

LEGO Resale Intelligence V2.

De evaluador de deals a motor de market intelligence. El salto estratégico, técnico y visual de V1 a V2.

PDF-READY · PORTFOLIO-GRADE · CODEX-DRIVEN

18 SEMANAS	3 CAPAS	6 VISTAS	FastAPI + Next.js
de ejecución	de inteligencia	Next.js premium	stack profesional

Perfil objetivo	Tiempo estimado	Stack
Quien terminó V1 · Ambición real	18 semanas · 5–8 h/semana	FastAPI · Next.js · PostgreSQL · LLM API

■ HONESTIDAD BRUTAL ANTES DE EMPEZAR

Esta guía asume que terminaste V1. No es para alguien que parte de cero. El stack de V2 (FastAPI + Next.js + PostgreSQL) es un salto real de complejidad. Codex lo genera — pero tú tienes que entenderlo. Si no puedes explicar cada módulo en una entrevista, frena y consolida antes de avanzar. Una V2 que no puedes defender es peor que una V1 impecable.

Índice

- A Resumen ejecutivo**
El salto de V1 a V2. Por qué esto diferencia más.
- B Visión del producto V2**
La nueva pregunta. Qué hace. Qué no hace. El pitch.
- C Alcance de V2**
Matriz IN/OUT. Qué viene de V1. Qué es nuevo.
- D Stack recomendado V2**
FastAPI, Next.js, PostgreSQL. Por qué cada decisión.
- E Arquitectura del sistema**
Backend desacoplado. Módulos. Endpoints. Estructura.
- F Modelo de datos**
Tablas nuevas. Modificadas. Relaciones. La clave del sistema.
- G Lógica de negocio**
Pricing dinámico. P&L.; Briefings IA. Hold/sell signals.
- H Diseño visual y UX/UI**
El salto a Next.js. 6 pantallas. Componentes. Design tokens.
- I Roadmap paso a paso**
7 fases, 18 semanas, entregable concreto por fase.
- J Plan de construcción con Codex**
8 prompts plantilla para los módulos clave.
- K Entregables finales**
Los 9 artefactos que convierten V2 en pieza de portfolio.
- L Cómo contar el proyecto**
CV. LinkedIn. Entrevista. Adaptar al rol.
- M Riesgos y errores a evitar**
Lo que destruye un proyecto ambicioso de portfolio.

Resumen ejecutivo

V1 demostró que sabes construir: pipeline real, lógica de negocio, UI premium, deploy público, case study. Eso es mucho. Pero V1 responde una pregunta estrecha: **¿es este listing concreto un buen trato?** No sabe nada del mercado. Sus precios son manuales. No detecta tendencias. No rastrea lo que ya tienes. Espera a que tú traigas un link.

★ **"Convierte el mercado de LEGO retirado en inteligencia estructurada: pricing dinámico, tendencias por categoría, briefings de mercado generados por IA y tracking de portfolio con P&L; real."**

La pregunta que cambia todo

V1: ¿Es este listing un buen trato? — reactivo, puntual, ciego al mercado.
V2: ¿Qué está pasando en el mercado, qué debería comprar y cuándo debería vender lo que ya tengo? — proactivo, temporal, con memoria.

V1 vs V2 — La diferencia en una tabla

V1	V2
1 URL → 1 análisis (reactivo)	Mercado monitorizado + análisis on-demand
Precios manuales en CSV (10 sets)	Motor de pricing dinámico (300+ sets)
IA como extractor de texto	IA como analista de mercado + briefings
Sin historial de mercado	Historial de precios con tendencias
Sin portfolio	Portfolio tracker con P&L; real
Streamlit (monolito)	FastAPI + Next.js (desacoplado)
SQLite local	PostgreSQL (persistente, time-series)
Score puntual 0–100	Score + contexto de mercado + anomalía
Fair price fijo y manual	Fair price calculado con datos históricos
1 fuente de datos	Motor de datos + BrickLink price guide

Qué demuestra V2 además de lo que demostró V1

Dimensión	Por qué importa para el portfolio
Visión estratégica de producto	Entendiste el límite de V1 y lo convertiste en la oportunidad de V2. No quisiste hacer más features — quisiste cambiar la pregunta.
Arquitectura profesional real	Backend desacoplado de frontend. API documentada. ORM con migraciones. Esto es lo que se espera en cualquier empresa de producto.
IA con propósito real	No usas IA para 'añadir IA'. La usas para sintetizar inteligencia de mercado que ningún humano podría generar manualmente en menos de horas.
Pensamiento financiero aplicado	P&L, cost basis, unrealized gains, hold/sell signals. Esto diferencia a un PM que entiende el negocio del que no.
Salto de diseño demostrable	De Streamlit (que cualquier data scientist monta en un día) a Next.js + TailwindCSS (que parece un producto real de SaaS).

Números objetivo de V2

Métrica	Detalle
18 semanas	Duración estimada (5–8h/semana)
300+ sets	Catálogo con pricing dinámico
6 pantallas	Next.js — Market Overview, Scanner, Portfolio, Set Intelligence, Briefings, Analyze
3 capas IA	Extracción (V1 heredada) + Detección de anomalías + Briefings de mercado
1 API pública	FastAPI con docs automáticas — linkeable en el README
~0€ hosting	Railway free tier (backend) + Vercel free tier (frontend)

BLOQUE B

Visión del producto V2

Antes de tocar código, la visión tiene que estar cristalina. V2 no es 'V1 con más features'. Es un cambio de pregunta, de arquitectura y de narrativa.

Qué hará exactamente la V2

#	Función	Qué aporta vs V1
1	Monitorizar el mercado	Mantener un histórico de fair prices por set y condición. Detectar tendencias: qué sube, qué baja, qué categorías están activas.
2	Calcular fair price dinámico	Sustituir la tabla manual de V1 por un motor que pondera datos históricos y la guía de precios de BrickLink. Precio actualizable, no fijo.
3	Analizar listings con contexto de mercado	Como V1, pero ahora el score se enriquece con contexto: 'este listing está un 28% por debajo de la media de los últimos 30 días.'
4	Detectar anomalías de precio	Identificar automáticamente listings cuyo precio se desvía significativamente del fair price histórico — oportunidades inusuales o precios sospechosos.
5	Trackear portfolio con P&L; real	Registrar qué compraste, a qué precio, cuándo. Calcular P&L; no realizado (basado en fair price actual) y realizado (cuando vendes).
6	Generar briefings de mercado con IA	Claude Sonnet sintetiza los movimientos del mercado en un briefing ejecutivo semanal: qué movió, qué oportunidades hay, qué recomienda para el portfolio.
7	Recomendar hold/sell	Para cada posición del portfolio: señal basada en tendencia de precio, tiempo de holding y estacionalidad del set.

Qué NO hará V2 (explícito y consciente)

Excluido	Razón
Compra automática	Igual que V1. Ético y legal.
Scraping masivo en tiempo real	ToS, riesgo, ruido. BrickLink price guide es suficiente.
Multi-usuario / autenticación real	Single user. Auth añade 3 semanas sin valor de portfolio.
App móvil nativa	Next.js es responsive. Suficiente.
Modelo ML propio de pricing	Claude Sonnet hace el trabajo pesado. No reinventes.
Publicación automática en marketplaces	Otro producto. No el V2.
Alertas push / notificaciones	Fuera de scope. Post-V2 si el sistema valida.

Inputs y outputs V2

INPUTS	OUTPUTS
• URL de listing (on-demand, como V1) • Datos de BrickLink price guide (JSON curado / script) • Registros manuales de portfolio (compra/venta) • Catálogo CSV expandido (300+ sets) • Parámetros de coste (comisiones, envío)	• Ficha de listing con score + contexto de mercado • Precio dinámico con banda de confianza • Anomaly score: desviación vs media histórica • Dashboard de portfolio con P&L; (realizado + no realizado) • Briefing semanal de mercado generado por IA • Señal hold/sell por posición • Gráficos de tendencia por set / categoría

Valor entregado

Para ti como usuario: **reducir de horas a segundos** el tiempo de análisis del estado de tu portfolio y de las oportunidades activas del mercado. Ya no necesitas recordar referencias de precio — el sistema las mantiene. Ya no necesitas calcular tu P&L; en Excel — el sistema lo hace en tiempo real.

Para tu portfolio: demostrar que pasaste de 'construir herramientas' a 'diseñar sistemas de inteligencia'. Eso es lo que contrata un PM de producto data-driven.

Alcance de V2 — qué entra y qué no

El error más frecuente al pasar de V1 a V2 es querer meter todo lo que quedó fuera de V1. No. V2 tiene un scope propio, igualmente acotado. La diferencia es que el scope es más ambicioso en la dirección correcta: más inteligencia, mejor arquitectura, mejor diseño. No más fuentes, no más automatización.

Matriz IN / OUT

Categoría	IN — V2	OUT — post-V2
Fuentes de datos	BrickLink price guide (JSON curado) + V1 comparables migrados	APIs de eBay/Wallapop en tiempo real, Vinted, Catawiki
Catálogo	300+ sets retirados con datos completos	Catálogo completo Brickset (miles de sets), sync automático
Captura de listings	URL manual on-demand (como V1, mejorada)	Crawler programado, alertas automáticas
Pricing	Motor dinámico con histórico y tendencias	Modelo predictivo ML propio, forecasting
Portfolio	Tracking manual de compras/ventas, P&L; calculado	Integración con cuentas de marketplace, sync automático
Briefings IA	Generación on-demand + semanal (Claude Sonnet)	Multi-agente, análisis de sentimiento de foros
Frontend	Next.js + TailwindCSS · 6 vistas · desktop-first responsive	App móvil nativa, PWA completa
Backend	FastAPI · endpoints documentados · single user	Auth JWT, multi-tenant, rate limiting de producción
DB	PostgreSQL local + Railway (cloud)	Redis cache, Elasticsearch, time-series DB dedicado
Deploy	Railway (backend) + Vercel (frontend)	Docker swarm, Kubernetes, CI/CD completo

Qué se reutiliza de V1 (no reescribir lo que ya funciona)

Módulo V1	Decisión	Notas
capture.py	Sí reutilizar	Mover a backend/core/. Puede necesitar ajustes menores.
extract.py	Sí reutilizar	El prompt LLM para extracción sigue siendo válido. Mejorar el schema.
identify.py	Sí reutilizar	Matching contra catálogo. Actualizar para catálogo expandido.

scoring.py	ADAPTAR	Los pesos base se mantienen. Añadir contexto de mercado y anomaly score.
models.py	AMPLIAR	Añadir nuevas tablas. Migrar con Alembic en lugar de init_db manual.
db.py	REESCRIBIR	Migrar de SQLite a PostgreSQL. Separar en database.py + crud.py.
app/ (Streamlit)	DEPRECAR	El frontend Streamlit se sustituye por Next.js. Se puede mantener como legacy.
data/catalog.csv	EXPANDIR	Actualizar a 300+ sets con campos adicionales.
tests/	MIGRAR Y AMPLIAR	Los tests de scoring/pricing siguen siendo válidos. Añadir tests de API.

NOTA SIMULACIÓN INTELIGENTE EN V2

Los datos de BrickLink price guide no se van a scrapear en tiempo real. En las primeras fases, se curan manualmente en un JSON estructurado para los 50 sets más activos. Esto es correcto: primero validas que el motor de pricing funciona, luego automatizas la fuente. En entrevistas: 'empecé con datos curados para validar el modelo antes de automatizar la ingesta.'

Lo que es manual / automático en V2

Paso	Modo V2	Por qué
Captura de precio BrickLink	Curado en JSON manualmente (fases 0–2), scripted después	Valida modelo antes de automatizar
Captura de listings	Manual URL on-demand	Como V1, mismo razonamiento
Registro de portfolio	Manual (compras/ventas que introduces tú)	Tú eres el single user
Generación de briefing	On-demand + cron semanal opcional	IA genera, tú decides cuándo
Score de oportunidad	Automático (mejorado con contexto)	Fórmula explícita
Señal hold/sell	Automático (lógica de negocio transparente)	No caja negra
Decisión final de compra	Manual humana siempre	El sistema ayuda, no decide

Stack recomendado V2

El stack de V2 es el salto más visible del proyecto. De monolito Streamlit a arquitectura desacoplada profesional. Cada decisión está justificada y es defendible en entrevista. No es over-engineering: es el stack mínimo que permite demostrar que sabes construir como lo hace una empresa real.

Stack completo V2

Capa	Tecnología	Por qué (defensa en entrevista)
Backend · API	FastAPI (Python 3.12+)	API REST con docs automáticas (Swagger/Redoc). Async nativo. Pydantic v2 integrado. Estándar en empresas de producto data-driven.
Frontend · UI	Next.js 14 + TailwindCSS	App Router. Server components. El stack estándar para SaaS moderno. Recharts para visualizaciones. Shadcn/ui para componentes base.
Base de datos	PostgreSQL 16	Time-series con tablas de histórico de precios. Relaciones sólidas. Railway free tier. Migrar desde SQLite con Alembic.
ORM + Migraciones	SQLAlchemy 2.0 + Alembic	Mismo ORM que V1, con migraciones gestionadas. Alembic genera el historial de cambios de esquema — profesional y auditable.
LLM · Extracción	Anthropic Claude Haiku (heredado de V1)	Para extracción estructurada de listings. Rápido y barato.
LLM · Inteligencia	Anthropic Claude Sonnet	Para briefings de mercado y análisis de anomalías. Más capacidad de razonamiento que Haiku — necesaria para síntesis compleja.
Datos de precio	BrickLink price guide (JSON curado)	Fuente más fiable de precios de mercado LEGO. Manual en fases tempranas, scriptable después.
Validación de datos	Pydantic v2	Mismo que V1. Modelos de API request/response + modelos de DB.
Testing	Pytest + httpx (async)	Tests unitarios (scoring, pricing engine) + tests de integración de API endpoints.
Gestión de paquetes	uv	Reemplaza venv + pip. Más rápido, más moderno. Fácil de explicar.
Control de versiones	Git + GitHub público	Igual que V1. Cada commit cuenta la historia del proyecto.
Deploy backend	Railway	PostgreSQL + FastAPI en Railway free tier. Un solo comando de deploy.
Deploy frontend	Vercel	Next.js en Vercel es el match natural. CI/CD automático desde GitHub.

El salto clave: de Streamlit a FastAPI + Next.js

En V1, Streamlit era la elección correcta. Permitía tener UI + backend en un solo archivo, sin curva de aprendizaje adicional. En V2, Streamlit tiene un techo: no puedes construir un dashboard que se parezca a un producto real de SaaS, y mezclar lógica de negocio con presentación en un monolito no escala bien.

FastAPI separa el backend (lógica, datos, IA) del frontend (presentación, UX). Next.js permite construir interfaces que se ven como productos reales. La combinación demuestra que entiendes arquitectura, no solo scripting.

★ CÓMO DEFENDER EL STACK EN ENTREVISTA

Pregunta habitual: '¿Por qué Next.js y no seguir con Streamlit?' Respuesta calibrada: 'Streamlit es excelente para prototipos y dashboards internos. V1 lo demuestra. Pero V2 requería separar backend de frontend para poder tener una capa de API reutilizable, y necesitaba un frontend sin el techo visual de Streamlit. Next.js + FastAPI es el stack estándar para productos SaaS modernos, y quería demostrar que puedo trabajar con él, no solo con librerías de data science.'

Por qué NO usar (todavía)

Tecnología	Por qué no ahora
Django	Demasiado opinionado para una API. FastAPI es más moderno y rápido de aprender.
React sin Next.js	Next.js añade routing, SSR y optimizaciones sin coste de setup.
MySQL	PostgreSQL es estándar en empresas de producto. MySQL es legacy en muchos contextos.
Redis / Celery	No necesitas cola de tareas en V2. APScheduler para el cron semanal es suficiente.
Docker en desarrollo	Añade complejidad sin valor en un proyecto solo. En deploy, Railway lo gestiona.
Embeddings / RAG	No tienes suficientes datos para justificarlo. Claude Sonnet es suficiente.
TypeScript estricto al 100%	Añade fricción. Usa TypeScript pero sin obsesionarte con strict mode en V2.

Costes estimados V2

Concepto	Coste estimado	Nota
Railway (backend + PostgreSQL)	~0–5€/mes	Free tier suficiente para portfolio. Hobby plan si necesitas persistencia.
Vercel (frontend)	0€	Free tier. Ilimitado para proyectos personales.
Anthropic API (Haiku)	~3–5€	Extracción de listings. Haiku es muy barato.
Anthropic API (Sonnet)	~5–10€	Briefings. ~100 briefings al mes = trivial.
Dominio (opcional)	~10–15€/año	Nice to have. Iri-intelligence.com o similar.

TOTAL V2	~10–20€ todo incluido	Comparable a V1. La diferencia está en la arquitectura, no en el coste.
----------	-----------------------	---

BLOQUE E

Arquitectura del sistema

V2 adopta una arquitectura desacoplada: backend REST (FastAPI) separado del frontend (Next.js). Esto permite que cada capa evolucione independientemente, que la API sea consumible desde cualquier cliente, y que el código sea más limpio y testable. No es over-engineering para V2 — es el mínimo de seriedad arquitectónica.

Diagrama de flujo del sistema

```
[ Frontend – Next.js / Vercel ]
↑ HTTP REST (JSON)
[ Backend – FastAPI / Railway ]
■■■■ /api/analyze → pipeline de extracción + scoring (heredado V1)
■■■■ /api/market → precios dinámicos, tendencias, anomalías
■■■■ /api/portfolio → holdings, P&L;, señales hold/sell
■■■■ /api/briefings → generación y consulta de briefings IA
■■■■ /api/watchlist → sets monitorizados
↑↓
[ PostgreSQL – Railway ]
■■■■ set_catalog (expandido: 300+ sets)
■■■■ price_history (NEW: historial temporal de precios)
■■■■ listings (V1 + nuevos campos)
■■■■ portfolio_items (NEW)
■■■■ transactions (NEW)
■■■■ market_briefings (NEW)
■■■■ watchlist (NEW)
↑↓
[ Anthropic API ]
■■■■ Claude Haiku → extracción estructurada de listings
■■■■ Claude Sonnet → briefings de mercado + análisis de anomalías
```

Estructura de carpetas recomendada

```

lego-resale-intelligence-v2/
  README.md
  .gitignore
  .env.example
  backend/ ← FastAPI app
    main.py ← entry point, CORS, routers
    requirements.txt
    routers/
      analyze.py ← POST /api/analyze
      market.py ← GET /api/market/price/{id}, trends
      portfolio.py ← CRUD /api/portfolio
      briefings.py ← GET/POST /api/briefings
      watchlist.py ← CRUD /api/watchlist
    services/
      price_engine.py ← motor de pricing dinámico
      ai_analyst.py ← briefings + anomaly detection (Sonnet)
      portfolio_engine.py ← P&L, hold/sell signals
      trend_detector.py ← tendencias de mercado
    core/ ← heredado de V1 (refactorizado)
      capture.py
      extract.py ← Haiku extractor
      identify.py
      scoring.py ← mejorado con contexto de mercado
    db/
      database.py ← engine, session, Base
      models.py ← SQLAlchemy models (todos)
      crud.py ← operaciones CRUD
      migrations/ ← Alembic
      alembic.ini
    tests/
      test_pricing.py
      test_scoring.py
      test_portfolio.py
      test_api.py ← httpx async tests
  frontend/ ← Next.js app
    package.json
    next.config.js
    tailwind.config.js
    src/
      app/ ← App Router
        page.tsx ← Market Overview (home)
        portfolio/page.tsx
        opportunities/page.tsx
        sets/[id]/page.tsx ← Set Intelligence
        briefings/page.tsx
        analyze/page.tsx
      components/
        ui/ ← shadcn/ui components
        charts/ ← Recharts wrappers
        layout/ ← Navbar, Sidebar, Layout
      lib/
        api.ts ← API client (fetch wrappers)
        types.ts ← TypeScript types compartidos
    data/
      catalog_v2.csv ← 300+ sets retirados
      bricklink_prices/ ← JSONs de precio por set

```

Endpoints de la API — resumen

Método	Endpoint	Descripción
POST	/api/analyze	Analiza una URL de listing. Devuelve score + fair price + contexto de mercado.
GET	/api/market/price/{set_id}	Fair price actual + histórico + tendencia para un set.
GET	/api/market/trends	Top sets por movimiento de precio (7d, 30d).
GET	/api/market/opportunities	Listings activos con score >= umbral.
GET	/api/portfolio	Todas las posiciones actuales con P&L; calculado.
POST	/api/portfolio	Añadir nueva posición (compra registrada).
PUT	/api/portfolio/{id}	Actualizar posición (venta, estado).
GET	/api/briefings	Historial de briefings generados.
POST	/api/briefings/generate	Genera un nuevo briefing con Claude Sonnet.
GET/POST/DELETE	/api/watchlist	CRUD de sets monitorizados.
GET	/api/sets/{set_id}	Datos completos de un set del catálogo.
GET	/docs	Swagger UI auto-generado por FastAPI (gratis).

★ LA DOCUMENTACIÓN AUTOMÁTICA ES UN ACTIVO DE PORTFOLIO

FastAPI genera /docs (Swagger) y /redoc automáticamente desde el código. Esto es linkeable en el README y en LinkedIn. No necesitas escribir la documentación de la API — solo escribir el código correctamente con type hints y docstrings.

BLOQUE F

Modelo de datos V2

V2 añade 5 tablas nuevas y expande 2 de V1. El corazón del sistema son **price_history** (inteligencia temporal) y **portfolio_items** (tu posición económica). Sin estas dos tablas, V2 sería solo V1 con mejor stack. Con ellas, es un sistema diferente.

Tabla: price_history (NUEVA — el corazón de V2)

Almacena el historial de precios de mercado por set y condición con timestamp. Esta tabla convierte el sistema de estático a temporal — la base de todas las tendencias.

Campo	Tipo	Descripción
id	INT (PK)	Autoincremental
set_id	STRING (FK)	→ set_catalog
condition	ENUM	SEALED · USED_COMPLETE · USED_INCOMPLETE
source	STRING	'bricklink_guide' · 'manual' · 'scrape'
price_min_eur	FLOAT	Mínimo del rango en la muestra
price_avg_eur	FLOAT	Media — este es el fair_price principal
price_max_eur	FLOAT	Máximo del rango
sample_count	INT	Nº de listings en que se basó la muestra
recorded_at	DATETIME	Cuándo se registró este snapshot de precio
is_stale	BOOL	True si tiene más de 14 días (flag de aviso en UI)

Tabla: portfolio_items (NUEVA)

Cada fila es un set que tienes o tuviste. Base del tracking de P&L.;

Campo	Tipo	Descripción
id	INT (PK)	Autoincremental
set_id	STRING (FK)	→ set_catalog
condition	ENUM	Condición en que lo compraste
quantity	INT	Unidades (normalmente 1)
cost_basis_eur	FLOAT	Precio total pagado (asking + envío recibido)

purchased_at	DATETIME	Cuándo compraste
listing_id	INT (FK, nullable)	Si vino de un listing analizado por el sistema
status	ENUM	HOLDING · LISTED · SOLD
notes	TEXT	Notas manuales
sell_target_eur	FLOAT (nullable)	Precio objetivo de venta

Tabla: transactions (NUEVA)

Historial completo de operaciones de compra y venta. Audit trail financiero.

Campo	Tipo	Descripción
id	INT (PK)	Autoincremental
portfolio_item_id	INT (FK)	→ portfolio_items
type	ENUM	BUY · SELL
gross_price_eur	FLOAT	Precio bruto de la operación
platform_fees_eur	FLOAT	Comisiones de la plataforma
shipping_eur	FLOAT	Coste de envío
net_eur	FLOAT	Resultado neto (calculado)
marketplace	STRING	Plataforma donde ocurrió
transacted_at	DATETIME	Fecha de la operación
notes	TEXT	Referencia o notas adicionales

Tabla: market_briefings (NUEVA)

Almacena los briefings generados por Claude Sonnet. Construye un historial de inteligencia.

Campo	Tipo	Descripción
id	INT (PK)	Autoincremental
period_start	DATE	Inicio del período analizado
period_end	DATE	Fin del período analizado
briefing_text	TEXT	Texto completo generado por Claude Sonnet
key_opportunities	JSON	Lista de set_ids destacados en el briefing
market_trend	ENUM	BULLISH · NEUTRAL · BEARISH

generated_at	DATETIME	Cuándo se generó
model_version	STRING	Versión del modelo usado (para trazabilidad)
token_count	INT	Tokens consumidos (para control de costes)

Tabla: watchlist (NUEVA)

Sets o categorías que monitorizas activamente.

Campo	Tipo	Descripción
id	INT (PK)	Autoincremental
set_id	STRING (FK, nullable)	Si monitorizas un set concreto
category	STRING (nullable)	Si monitorizas una categoría ('Star Wars UCS')
target_price_eur	FLOAT (nullable)	Precio objetivo de compra
alert_on_drop	BOOL	True = alertar cuando el precio baje del target
notes	TEXT	Por qué estás monitorizando esto
added_at	DATETIME	Cuándo lo añadiste

Tablas modificadas de V1

Tabla	Campo nuevo	Por qué
set_catalog	weight_kg (FLOAT)	Para calcular coste real de envío
set_catalog	minifigure_count (INT)	Factor de valor para coleccionistas
set_catalog	exclusive (BOOL)	Sets exclusivos (SDCC, Fan Vote, etc.) — mayor volatilidad
set_catalog	target_audience (ENUM)	KIDS · ADULTS · COLLECTORS — afecta a tendencias
listings	market_context_json (JSON)	Snapshot de price_history en el momento del análisis
listings	anomaly_score (FLOAT)	Desviación % del asking_price vs fair_price histórico
listings	ai_recommendation (ENUM)	BUY · WATCH · SKIP · INVESTIGATE
listings	market_trend_at_capture (ENUM)	Tendencia del set cuando se analizó

NOTA POR QUÉ ALEMBIC IMPORTA

V1 usaba un script `init_db.py` que creaba las tablas desde cero. En V2, con Alembic, cada cambio de schema genera una migración con timestamp. Puedes revertir cambios, ver la historia del schema, y deployar cambios de DB de forma segura. En una entrevista: 'gestioné las migraciones con Alembic — puedo explicar cualquier cambio de schema y cuándo ocurrió.'

Lógica de negocio V2

El cerebro del sistema. V2 amplía la lógica de V1 en cuatro direcciones: pricing dinámico (vs manual), detección de anomalías, cálculo de P&L; de portfolio, y generación de inteligencia narrativa con IA. Todo explícito, todo trazable.

1. Motor de pricing dinámico

Sustituye la tabla estática de comparables de V1. El fair_price se calcula ponderando tres fuentes con pesos configurables:

```
# price_engine.py
def compute_fair_price(set_id: str, condition: str) -> FairPrice:
    # Obtener los últimos N snapshots de price_history
    recent = get_recent_snapshots(set_id, condition, days=90)

    if not recent:
        return FairPrice(avg=None, min=None, max=None, confidence='LOW')

    # Weighted average: más peso a datos recientes
    weights = exponential_decay_weights([s.recorded_at for s in recent])
    weighted_avg = sum(s.price_avg_eur * w for s, w in zip(recent, weights))

    # Trend adjustment: si el precio está subiendo, el fair price sube
    trend_adj = compute_trend_adjustment(set_id, condition)
    fair_price = weighted_avg * (1 + trend_adj)

    confidence = 'HIGH' if len(recent) >= 5 else 'MEDIUM' if len(recent) >= 2 else 'LOW'
    return FairPrice(avg=fair_price, min=min(s.price_min_eur for s in recent),
                    max=max(s.price_max_eur for s in recent),
                    confidence=confidence, sample_count=len(recent))
```

2. Anomaly Score

Mide qué tan inusual es el precio de un listing respecto al histórico. Un anomaly_score negativo grande es una oportunidad. Uno positivo grande es sospechoso.

```
def compute_anomaly_score(asking_price: float, fair_price: float) -> float:
    # Positivo = asking está SOBRE el fair price (caro)
    # Negativo = asking está BAJO el fair price (oportunidad)
    return (asking_price - fair_price) / fair_price

# Umbrales:
# anomaly < -0.25 -> 'Precio muy por debajo del mercado - investigar'
# -0.25 <= anomaly < -0.10 -> 'Oportunidad interesante'
# -0.10 <= anomaly <= 0.10 -> 'Precio en rango normal'
# anomaly > 0.10 -> 'Caro vs mercado'
# anomaly > 0.30 -> 'Sobrevalorado significativamente'
```

3. Cálculo de P&L; del portfolio

Para cada posición del portfolio, el sistema calcula el estado económico en tiempo real, usando el fair_price dinámico como referencia de valor actual:

```
# portfolio_engine.py
FEES_RATE = 0.10 # comisión de plataforma al vender
SHIPPING_OUT = 8.0 # envío estimado al revender

def compute_position_pnl(item: PortfolioItem) -> PositionPnL:
    fair = price_engine.compute_fair_price(item.set_id, item.condition)
    current_value = fair.avg if fair.avg else item.cost_basis_eur

    # P&L; no realizado (si vendieras hoy al fair price)
    gross_if_sold = current_value
    net_if_sold = gross_if_sold * (1 - FEES_RATE) - SHIPPING_OUT
    unrealized_pnl = net_if_sold - item.cost_basis_eur
    unrealized_pnl_pct = unrealized_pnl / item.cost_basis_eur * 100

    days_held = (datetime.now() - item.purchased_at).days
    annualized_return = (unrealized_pnl_pct / days_held * 365) if days_held > 0 else 0

    return PositionPnL(
        cost_basis=item.cost_basis_eur,
        current_fair_value=current_value,
        unrealized_pnl_eur=unrealized_pnl,
        unrealized_pnl_pct=unrealized_pnl_pct,
        annualized_return_pct=annualized_return,
        days_held=days_held,
        price_confidence=fair.confidence,
    )
```

4. Señal Hold/Sell

Combina tendencia de precio, tiempo de holding y estacionalidad para dar una recomendación sobre si mantener o vender una posición:

```

def compute_hold_sell_signal(item: PortfolioItem) -> HoldSellSignal:
    trend = trend_detector.get_trend(item.set_id, item.condition, days=30)
    days_held = (datetime.now() - item.purchased_at).days

    # Factor de tendencia: sube el mercado?
    trend_score = {
        'STRONG_UP': 90, 'UP': 70, 'FLAT': 50,
        'DOWN': 30, 'STRONG_DOWN': 10
    }.get(trend.direction, 50)

    # Factor de holding: sets populares se venden bien hasta 6 meses
    # Después de 6 meses, el costo de oportunidad pesa más
    days_factor = max(0, 100 - (days_held / 180) * 40)

    # Estacionalidad: UCS Star Wars picos en Nov-Dic
    season_boost = get_seasonality_boost(item.set_id) # -10 a +20

    hold_score = (trend_score * 0.50 + days_factor * 0.30 + (50 + season_boost) * 0.20)

    if hold_score >= 65:
        signal = 'HOLD'
        reason = f'Mercado {trend.direction}, buenas condiciones para esperar'
    elif hold_score >= 45:
        signal = 'NEUTRAL'
        reason = 'Mercado plano. Evalúa tu necesidad de liquidez'
    else:
        signal = 'SELL'
        reason = 'Mercado bajista o holding prolongado. Considera vender'

    return HoldSellSignal(signal=signal, score=hold_score, reason=reason)

```

5. Generación de briefings con Claude Sonnet

El sistema agrega datos del mercado y los pasa a Claude Sonnet para síntesis. El resultado es un briefing ejecutivo con contexto, números concretos y recomendaciones:

```

# ai_analyst.py
def generate_market_briefing(period_days: int = 7) -> MarketBriefing:
# 1. Agregar datos del periodo
price_changes = get_price_changes(days=period_days) # top movers
opportunities = get_current_opportunities(score_min=65)
portfolio_snap = get_portfolio_snapshot()
trend_summary = trend_detector.get_market_summary()

# 2. Construir contexto para el LLM
context = {
'period': f'últimos {period_days} días',
'price_changes': price_changes[:10], # top 10 movers
'opportunities': opportunities[:5], # top 5 oportunidades activas
'portfolio': portfolio_snap,
'market_trend': trend_summary,
}

# 3. Llamada a Claude Sonnet
response = anthropic_client.messages.create(
model='claude-sonnet-4-6',
max_tokens=1200,
system=BRIEFING_SYSTEM_PROMPT,
messages=[{'role': 'user', 'content': json.dumps(context)}]
)

# 4. Parsear y persistir
briefing_text = response.content[0].text
market_tone = extract_market_tone(briefing_text) # regex simple
return persist_briefing(briefing_text, context, market_tone)

```

★ EL BRIEFING SYSTEM PROMPT — DISEÑARLO BIEN ES EL TRABAJO

El system prompt de generación de briefings es donde más diferencias se notan. Debe especificar: tono ejecutivo y directo, estructura fija (resumen / movimientos / oportunidades / portfolio / acción recomendada), longitud máxima (400 palabras), prohibición de información inventada ('si no tienes datos suficientes, indícalo'), y formato markdown. Pídele a Codex que lo genere y refínalo tú iterando sobre outputs reales.

Diseño visual y UX/UI

V2 es el salto de diseño más visible del proyecto. De Streamlit — que cualquier data scientist monta en un día — a Next.js con diseño de producto real. La primera impresión de quien vea el portfolio cambia completamente. Esto no es cosmético: es estratégico.

Filosofía de diseño V2

V1 se inspiró en Linear, Vercel, Stripe. V2 mantiene la misma inspiración pero la ejecuta con Next.js + TailwindCSS, lo que elimina las restricciones de Streamlit. El objetivo: que alguien que lo vea por primera vez piense 'esto parece un producto real', no 'esto es un proyecto de portfolio'.

Paleta cromática V2 (continuidad con V1)

Nombre	Hex	Uso
Ink	#0a0e27	Background oscuro. Mismo que V1. Coherencia visual.
LEGO Yellow	#fbbf24	Acento principal. CTAs, scores altos, highlights.
Surface	#f8fafc	Background claro. Modo light.
Signal Up	#10b981	Verde. P&L; positivo, tendencia alcista, GREEN score.
Signal Down	#ef4444	Rojo. P&L; negativo, alerta, RED score.
Accent Blue	#3b82f6	Nuevo en V2. Links, información neutral, NEUTRAL signal.
Slate	#64748b	Texto secundario, labels, metadatos.
Amber	#f59e0b	Warnings, AMBER score, señales de atención.

Las 6 pantallas de V2

1. Market Overview (Home)

El pulso del mercado en tiempo real. Primera pantalla que ve el usuario.

- Market Trend Badge: BULLISH / NEUTRAL / BEARISH con flecha y color
- 6 KPI cards: oportunidades activas, margen promedio, valor portfolio, P&L; no realizado, sets tracked, fecha último briefing
- 'Oportunidades destacadas': top 5 listings con score >= 70
- 'Movimientos de mercado': top 5 sets con mayor cambio de precio en 7 días
- 'Tu portfolio hoy': mini-resumen con P&L; total del día
- CTA: 'Ver briefing completo' → enlaza a Briefings

2. Opportunity Scanner

Análisis de listings on-demand. Evolución de la pantalla Analyze de V1.

- Input URL único o múltiple (textarea, una URL por línea)
- Queue visual: 'analizando...' con progreso animado
- Tabla de resultados ordenable: score, margen, anomaly, fecha
- Filtros: score mínimo, condición, tema, anomaly type
- Por cada listing: badge de score + margen pill + anomaly indicator + botón 'Añadir al portfolio'
- Contexto de mercado inline: '28% por debajo del precio histórico de 30 días'

3. Portfolio Tracker

Estado económico real de tus holdings.

- Header: portfolio total value + unrealized P&L; total (EUR + %)
- Tabla de posiciones: set, condición, coste, valor actual, P&L;, días en cartera, señal hold/sell
- Cada fila expandible: detalle de P&L;, histórico de precio del set, botón 'Registrar venta'
- Gráfico: evolución del valor de portfolio en el tiempo (Recharts LineChart)
- Sección 'Posiciones cerradas': historial de ventas con P&L; realizado
- Botón 'Añadir posición manual' para registros sin listing previo

4. Set Intelligence (/sets/[id])

Deep dive en un set concreto. La pantalla más analítica.

- Header: imagen del set + nombre + set_id + año de retiro + retail price
- Gráfico de precio histórico interactivo: 30d / 90d / 180d (Recharts AreaChart)
- Precio actual: MIN / AVG / MAX con banda de confianza y fecha de último update
- Trend indicator: flecha + porcentaje + 'vs 30 días'
- '¿Cuándo comprar?': guía basada en estacionalidad y tendencia actual
- Listings recientes analizados para este set
- Botón 'Añadir a watchlist'

5. AI Briefings

Feed de inteligencia de mercado generada por Claude Sonnet.

- Briefing más reciente destacado: texto completo + metadata (período, tone, sets mencionados)
- Historial de briefings anteriores: cards con resumen + fecha + market tone badge
- Botón 'Generar nuevo briefing' (con confirmación de coste estimado en tokens)
- Cada briefing: markdown renderizado + botón copiar + highlight de sets mencionados

- Market tone timeline: gráfico de BULLISH/NEUTRAL/BEARISH a lo largo del tiempo

6. Analyze (heredada de V1, mejorada)

Análisis de una URL individual con contexto de mercado completo.

- Input URL + botón analizar
- Resultado: ficha completa con score + anomaly score + fair price dinámico
- Contexto de mercado: '¿Cómo está el mercado de este set ahora?' → sparkline de precio
- Recomendación IA: BUY / WATCH / SKIP / INVESTIGATE con razonamiento
- Botón 'Añadir al portfolio si lo compras'
- Historial de últimos análisis (como V1)

Componentes clave V2

Componente	Descripción
MarketTrendBadge	BULLISH / NEUTRAL / BEARISH con flecha, color y pulso animado sutil
PriceSparkline	Mini gráfico de línea (30d) inline en tablas. Recharts en miniatura.
PLPill	€X (+Y%) con color verde/rojo. Formato financiero. Siempre signo visible.
HoldSellSignal	HOLD / NEUTRAL / SELL con color, icono y tooltip de razonamiento
AnomalyBadge	% de desviación vs mercado. Rojo si caro, verde si oportunidad.
BriefingCard	Resumen de briefing con market tone, fecha y top oportunidades mencionadas
ConfidenceDot	Punto de color para precio: HIGH (verde) / MEDIUM (amber) / LOW (rojo)
ScoreBadge	Igual que V1 pero ahora con contexto: '82 · mercado alcista'
PortfolioPositionRow	Fila de tabla expandible con P&L; detallado y señal hold/sell
KPICard	Valor grande + label + delta. Sin iconos decorativos. Número tabular.

★ CÓMO CONSEGUIR DISEÑO PREMIUM EN NEXT.JS SIN SER DISEÑADOR

Tres herramientas: (1) shadcn/ui para los componentes base (botones, inputs, dialogs, tables) — copias el código, personalizas los colores con TailwindCSS. (2) Recharts para todas las visualizaciones — LineChart, AreaChart, BarChart. (3) Framer Motion para micro-animaciones en transiciones y estados de carga. Con estas tres, el resultado parece Linear sin necesidad de ser diseñador.

Roadmap paso a paso

18 semanas · 7 fases · cada fase con un entregable funcional concreto. Si una fase se atrasa más de una semana, recorta scope antes de extender plazos. El error número uno en V2 es querer hacer todo a la vez. No.

FASE 0 — Fundamentos y setup

Semana 1 · ~6h

Objetivo: Entorno listo, repo creado, estructura clara antes de escribir una línea de negocio.

- Crear repo GitHub público lego-resale-intelligence-v2
- Setup uv + requirements.txt (backend) + package.json (frontend)
- Inicializar FastAPI con main.py mínimo — que responda /health
- Inicializar Next.js con create-next-app (TypeScript, TailwindCSS, App Router)
- PostgreSQL local instalado. Railway account creada.
- Copiar módulos V1 a backend/core/ (capture, extract, identify, scoring)
- Crear .env.example con todas las variables necesarias
- Crear vision_v2.md en el repo con el pitch y la pregunta que V2 responde
- Primer commit: estructura vacía pero coherente

FASE 1 — Backend foundation — API + DB

Semanas 2–4 · ~20h

Objetivo: 'Una API que acepta una URL y devuelve un análisis completo, respaldada por PostgreSQL.'

- Setup SQLAlchemy 2.0 + Alembic. Primera migración: tablas de V1 en PostgreSQL
- Migrar models.py de V1 expandido (nuevos campos en listings y set_catalog)
- Migrar db.py → separar en database.py (engine/session) + crud.py (operaciones)
- Router analyze.py: POST /api/analyze funcional end-to-end
- Cargar catalog_v2.csv en DB (300 sets). Script de seed.
- Tests de integración: POST /api/analyze con 10 URLs reales
- FastAPI docs (/docs) funcionando y navegable
- Deploy inicial en Railway (backend + PostgreSQL). URL pública.

FASE 2 — Motor de pricing dinámico

Semanas 5–6 · ~14h

Objetivo: 'Fair prices calculados desde datos históricos, no desde una tabla estática.'

- Crear tabla price_history con Alembic migration
- Curar manualmente datos de BrickLink para los 50 sets más activos → JSON seed
- Script de ingesta: cargar JSON → price_history con timestamps correctos
- Implementar price_engine.py: weighted average + trend adjustment
- Router market.py: GET /api/market/price/{set_id} + GET /api/market/trends
- Integrar price_engine en el pipeline de analyze (reemplaza lookup estático)
- Añadir anomaly_score al resultado de analyze
- Tests: scoring con precio dinámico vs manual — verificar coherencia

FASE 3 — Capa de inteligencia IA

Semanas 7–8 · ~14h

Objetivo: 'Claude Sonnet genera briefings de mercado con datos reales.'

- Crear tabla market_briefings con Alembic migration
- Implementar ai_analyst.py: system prompt del briefing + llamada Sonnet
- Router briefings.py: GET /api/briefings + POST /api/briefings/generate
- Implementar trend_detector.py: cálculo de tendencias por set y categoría
- Integrar ai_recommendation en el pipeline de analyze
- Generar 3 briefings reales con datos de Fase 2. Revisar calidad. Ajustar prompt.
- Tests: mock de Sonnet response. Test de parseo y persistencia.

FASE 4 — Portfolio tracker

Semanas 9–10 · ~14h

Objetivo: 'Sé exactamente cuánto vale lo que tengo y si debo venderlo.'

- Crear tablas portfolio_items + transactions + watchlist con migraciones
- Implementar portfolio_engine.py: compute_position_pnl + hold_sell_signal
- Router portfolio.py: CRUD completo de posiciones y transacciones
- Router watchlist.py: CRUD de sets monitorizados
- Seed de demo: 15 posiciones de portfolio con fechas históricas reales
- Tests: P&L; con precios mock. Señales hold/sell con diferentes tendencias.

FASE 5 — Frontend Next.js

Semanas 11–15 · ~28h

Objetivo: 'Una app que parece un producto real, no un proyecto de portfolio.'

- Setup lib/api.ts: fetch wrapper tipado para todos los endpoints
- Setup lib/types.ts: tipos TypeScript desde las respuestas de FastAPI
- Layout: Navbar + Sidebar + estructura base de todas las páginas
- Página 1: Market Overview — KPIs, oportunidades, movimientos, portfolio snapshot
- Página 2: Opportunity Scanner — input URL, cola de análisis, tabla de resultados
- Página 3: Portfolio Tracker — tabla de posiciones, P&L; chart, posiciones cerradas
- Página 4: Set Intelligence — precio histórico chart, trend, contexto, watchlist
- Página 5: AI Briefings — feed de briefings, generación on-demand
- Página 6: Analyze — input URL individual (como V1 pero con contexto de mercado)
- Deploy frontend en Vercel. Conectar con backend Railway.
- Responsive: verificar todas las pantallas en móvil (768px mínimo)

FASE 6 — Polish, seed data y documentación

Semanas 16–17 · ~12h

Objetivo: 'De prototipo funcional a proyecto enseñable.'

- Seed completo: 6 meses de price_history para 50 sets, 20 portfolio positions, 8 briefings
- Estados vacíos diseñados: empty states, loading states, error states en todas las páginas
- Revisión de responsive en todas las pantallas
- README.md final: problema, solución, stack, arquitectura, capturas, demo, how to run
- Diagrama de arquitectura (Excalidraw o Figma) → docs/architecture.png
- 10 capturas pulidas desktop + 4 capturas mobile → docs/screenshots/
- Vídeo demo de 3 minutos (Loom): problema → V1 → qué faltaba → V2 → resultado
- Case study PDF actualizado (V1 → V2 evolution narrative)
- 3 bullets CV actualizados. Post LinkedIn redactado.

■ TRAMPA CRÍTICA DE V2: SALTARSE FASES 2 Y 4

La tentación es: montar el frontend bonito primero (Fase 5) sin tener el backend completo. Error fatal. El frontend sin datos reales es un mockup — no demuestra nada. Completa las Fases 1-4 antes de tocar Next.js. Cuando llegues a Fase 5, tendrás datos reales que mostrar y la diferencia será enorme.

Plan de construcción con Codex

Mismo principio que en V1: Codex es un colega senior, no un subcontratista. No pegas código que no entiendes. En V2 el riesgo es mayor porque el stack es más complejo y las interdependencias entre backend y frontend son reales. Domina cada módulo antes de pasar al siguiente.

■ REGLA DE ORO V2

Si no puedes explicar la diferencia entre un router FastAPI y un service, para. Si no puedes decir qué hace `useEffect` en Next.js, para. Pídele a Codex que te lo explique. V2 con partes que no entiendes es más peligroso en entrevista que una V1 que entiendes perfectamente.

PROMPT TEMPLATE · 1 · FASTAPI SETUP + ESTRUCTURA

```
Soy un desarrollador que viene de construir V1 en Python/Streamlit.
Voy a construir la V2 con FastAPI. Necesito:

1. main.py con: CORS configurado, routers registrados,
   lifespan para init de DB, /health endpoint
2. Estructura de routers: analyze, market, portfolio, briefings, watchlist
   Cada uno como archivo vacío con un endpoint de prueba GET /ping
3. database.py: engine PostgreSQL con SQLAlchemy 2.0 async,
   get_db dependency, Base declarativa
4. requirements.txt: fastapi, uvicorn, sqlalchemy[asyncio],
   asynpg, alembic, pydantic[email], anthropic, httpx, python-dotenv

Variables de entorno necesarias: DATABASE_URL, ANTHROPIC_API_KEY,
ANTHROPIC_MODEL_HAIKU, ANTHROPIC_MODEL_SONNET, CORS_ORIGINS

Restricciones:
- Async en todo: async def, await, AsyncSession
- Sin auth, sin rate limiting (V2 solo user)
- Comentarios en español
- Al final explícame la diferencia entre router y service en FastAPI
```

PROMPT TEMPLATE · 2 · ALEMBIC + MIGRACIONES

Necesito configurar Alembic para gestionar las migraciones de PostgreSQL.

Tengo estos modelos SQLAlchemy 2.0 (te los paso al final):

- SetCatalog, Listing, Comparable (de V1, expandidos)
- PriceHistory, PortfolioItem, Transaction, MarketBriefing, Watchlist (nuevos)

Necesito:

1. alembic init en backend/db/migrations/
2. alembic.ini configurado para leer DATABASE_URL del .env
3. env.py configurado con todos los modelos importados
4. Primera migración generada: alembic revision --autogenerate -m 'initial'
5. Instrucciones para: crear migración, aplicar, revertir

Restricciones:

- Async engine (AsyncEngine de sqlalchemy.ext.asyncio)
- Naming convention explícito para constraints
- Explícame al final cómo Alembic detecta cambios de schema

PROMPT TEMPLATE · 3 · MOTOR DE PRICING DINÁMICO

Necesito implementar backend/services/price_engine.py.

Schema de PriceHistory (SQLAlchemy):

set_id: str, condition: str, price_min/avg/max_eur: float,
sample_count: int, recorded_at: datetime, source: str

Necesito:

1. async def get_recent_snapshots(db, set_id, condition, days=90) -> list[PriceHistory]
2. def exponential_decay_weights(dates: list[datetime]) -> list[float]

Más peso a datos recientes. Suma de pesos = 1.

3. async def compute_fair_price(db, set_id, condition) -> FairPrice

Donde FairPrice es Pydantic con: avg, min, max, confidence ('HIGH'|'MEDIUM'|'LOW'),
sample_count, days_since_update

4. def compute_anomaly_score(asking: float, fair_avg: float) -> float

5. async def get_price_changes(db, days=7) -> list[PriceChange]

Top sets ordenados por mayor variación de precio en el período

Restricciones:

- Todo async con AsyncSession
- Si no hay datos, devolver FairPrice con confidence='NONE' (no lanzar error)
- Tests en tests/test_pricing.py con 5 casos: datos completos, datos escasos, sin datos, anomalía positiva, anomalía negativa

PROMPT TEMPLATE · 4 · AI BRIEFING — SYSTEM PROMPT + GENERACIÓN

Necesito implementar backend/services/ai_analyst.py.

Tarea principal: generar un briefing de mercado semanal sobre LEGO retirado.

Necesito:

1. BRIEFING_SYSTEM_PROMPT: string con instrucciones para Claude Sonnet.

El briefing debe tener estructura fija:

- Resumen ejecutivo (2 líneas)
- Movimientos de mercado (top 3 sets que más se movieron)
- Oportunidades activas (top 3 listings con mejor score)
- Estado del portfolio (P&L; no realizado total, señal recomendada)
- Acción recomendada esta semana (concreta, accionable)

Tono: ejecutivo, directo, sin relleno. Max 350 palabras. En español.

Si los datos son insuficientes para una sección, indícalo explícitamente.

2. async def generate_briefing(db, period_days=7) -> MarketBriefing:

- Agrega datos (price_changes, opportunities, portfolio_snap)
- Llama a claude-sonnet-4-6
- Parsea market_trend del texto (regex: busca BULLISH/NEUTRAL/BEARISH)
- Persiste en market_briefings
- Devuelve MarketBriefing (Pydantic)

3. async def detect_anomalies(db, threshold=0.25) -> list[AnomalyAlert]

Listings cuyo anomaly_score supere el umbral positivo o negativo

Restricciones:

- Manejo de error si Sonnet falla: loggear, no propagar (devolver briefing vacío con flag)
- Loggear tokens consumidos para control de costes
- Explicame al final cómo cambiarías el prompt para priorizar un tipo de set específico

PROMPT TEMPLATE · 5 · PORTFOLIO ENGINE — P&L; + HOLD/SELL

Necesito implementar backend/services/portfolio_engine.py.

Modelos disponibles: PortfolioItem, Transaction, PriceHistory.
price_engine.compute_fair_price() ya existe y funciona.

Necesito:

1. Pydantic: PositionPnL con campos:

cost_basis_eur, current_fair_value_eur, unrealized_pnl_eur,
unrealized_pnl_pct, annualized_return_pct, days_held,
price_confidence, hold_sell_signal, hold_sell_reason

2. async def compute_position_pnl(db, item: PortfolioItem) -> PositionPnL

Usa price_engine para current_fair_value.

net_if_sold = fair_value * (1 - 0.10) - 8.0 (fees + shipping)

unrealized_pnl = net_if_sold - cost_basis

3. def compute_hold_sell_signal(pnl, trend_direction: str, days_held: int) -> tuple[str, str]

Devuelve (signal: 'HOLD'|'NEUTRAL'|'SELL', reason: str)

Lógica según Bloque G de la guía.

4. async def get_portfolio_summary(db) -> PortfolioSummary

Total cost_basis, total current_value, total unrealized_pnl,

top_position (mayor P&L;), worst_position (menor P&L;)

Tests en tests/test_portfolio.py:

- Posición en profit fuerte → HOLD
- Posición neutral + mercado bajista → SELL
- Posición con precio sin datos → confidence=NONE

PROMPT TEMPLATE · 6 · NEXT.JS — SETUP + API CLIENT

Necesito configurar el proyecto Next.js 14 (App Router) para consumir mi FastAPI en Railway.

Necesito:

1. next.config.js: NEXT_PUBLIC_API_URL desde env, rewrites opcionales

2. tailwind.config.js: paleta custom con mis colores:

ink: '#0a0e27', lego-yellow: '#fbbf24', signal-up: '#10b981',
signal-down: '#ef4444', surface: '#f8fafc', accent: '#3b82f6'

3. lib/types.ts: interfaces TypeScript para:

AnalysisResult, FairPrice, PortfolioItem, PositionPnL,
MarketBriefing, PriceHistory, HoldSellSignal

4. lib/api.ts: fetch wrappers async para todos los endpoints:

analyzeUrl(), getMarketPrice(setId), getPortfolio(),
generateBriefing(), getLatestBriefing(), addPosition()

Manejo de errores: throw ApiError con status + message

5. app/layout.tsx: Navbar + Sidebar responsive (collapsed en móvil)

Stack adicional a instalar:

recharts, @radix-ui/react-*, class-variance-authority, lucide-react

Restricciones:

- Server components donde sea posible. Client components solo cuando necesario.
- loading.tsx en cada ruta para Suspense automático
- Sin Redux. useState + SWR para data fetching del lado cliente.
- Explicame cuándo usar 'use client' y cuándo no

PROMPT TEMPLATE · 7 · MARKET OVERVIEW — PANTALLA PRINCIPAL

Necesito implementar app/page.tsx: el Market Overview.

Datos que vienen de la API (ya disponibles):

- GET /api/market/trends → {top_movers: PriceChange[], market_trend: string}
- GET /api/portfolio (summary) → PortfolioSummary
- GET /api/market/opportunities?score_min=65 → AnalysisResult[]
- GET /api/briefings?limit=1 → MarketBriefing[]

Layout:

- MarketTrendBadge en header (BULLISH/NEUTRAL/BEARISH con color y flecha)
- 6 KPI cards en grid 3x2: [oportunidades activas, margen promedio, valor portfolio, P&L; no realizado, sets tracked, última fecha briefing]
- 'Top oportunidades': tabla compacta con score badge + margin pill + set name
- 'Movimientos de mercado': lista de top 5 movers con sparkline simple
- 'Portfolio hoy': mini card con P&L; total + señal dominante

Componentes a crear en components/ui/:

MarketTrendBadge, KPICard, ScoreBadge, MarginPill, PLPill

Restricciones:

- Server component: fetch en el server (no useEffect para los datos iniciales)
- Paleta: bg-surface en light, bg-ink en dark. Toggle de tema con next-themes.
- Skeleton loading para cada sección
- Responsive: 1 columna en móvil, grid en desktop
- Explicame la diferencia entre fetch en Server Component y SWR en Client

PROMPT TEMPLATE · 8 · PRICE HISTORY CHART — COMPONENTE RECHARTS

Necesito un componente React para el gráfico de precio histórico de la pantalla Set Intelligence.

Componente: components/charts/PriceHistoryChart.tsx

Props:

data: Array<{date: string, price_min: number, price_avg: number, price_max: number}>
setName: string
currentPrice?: number

Requisitos de diseño:

- AreaChart de Recharts con tres líneas: min, avg, max
- Area fill entre min y max con opacidad 0.15 (banda de confianza)
- Línea avg prominente en lego-yellow (#fbbf24)
- Líneas min/max en slate (#64748b) más finas
- Si currentPrice existe: línea horizontal de referencia (ReferenceLine)
- Tooltips personalizados: fecha + tres precios + desviación vs avg
- Selector de rango: botones 30d / 90d / 180d que filtran los datos
- Eje X: fechas en formato 'DD MMM'. Eje Y: '€X'
- Background: transparente. Grid sutil en slate-light.
- Responsive: usa ResponsiveContainer de Recharts

Restricciones:

- 'use client' (Recharts lo necesita)
- Sin dependencias adicionales más allá de recharts
- Accesible: aria-label en el chart
- Explicame qué es ResponsiveContainer y por qué es necesario

NOTA CÓMO VALIDAR LO QUE CODEX GENERA EN V2

V2 tiene una capa extra de complejidad: backend y frontend deben hablar entre sí. Validación mínima por cada módulo: (1) ¿El endpoint responde con la forma correcta? Compruébalo en /docs de FastAPI. (2) ¿El componente React recibe y renderiza los datos? Pruébalo con datos mock hardcodeados antes de conectar la API real. (3) ¿El test pasa? Si el test no tiene sentido, el test está mal, no el código.

BLOQUE K

Entregables finales V2

V2 tiene los mismos entregables que V1 más algunos adicionales que aprovechan la mayor sofisticación del sistema. La narrativa de portfolio evoluciona: ya no es 'construí una herramienta', es 'construí un sistema de inteligencia'.

#	Entregable	Contenido	Dónde
1	App V2 desplegada	Backend Railway + Frontend Vercel. Dos URLs públicas.	https://lri-v2.up.railway.app (API) + lri-v2.vercel.app (UI)
2	API documentada pública	Swagger en /docs. Linkeable desde README y LinkedIn.	https://lri-v2.up.railway.app/docs
3	Repo GitHub público	Commits con historia clara. Dos carpetas: backend/ + frontend/.	github.com/tu-usuario/lego-resale-intelligence-v2
4	README V2 impecable	Hero, problema, V1→V2 evolution, stack, arquitectura, capturas, demos, how to run.	Raíz del repo
5	Diagrama de arquitectura V2	Actualizado con la arquitectura desacoplada. FastAPI + Next.js + PostgreSQL.	docs/architecture_v2.png
6	Vídeo demo (3 min)	Estructura: problema (15s) → qué limitaba V1 (20s) → V2 en uso (120s) → resultado (25s).	Loom link en README
7	10 capturas pulidas	Las 6 pantallas + detalle de briefing + portfolio P&L; + Set Intelligence chart + mobile.	docs/screenshots/
8	Case study PDF V2	Narrativa de evolución: V1 → límites encontrados → decisiones de V2 → resultados.	docs/case_study_v2.pdf
9	Post LinkedIn	Narrativa de evolución (no de lanzamiento desde cero). Foco en las decisiones de producto.	LinkedIn

Plantilla README V2 (estructura)

```

# LEGO Resale Intelligence V2
> De evaluador de deals a motor de market intelligence.
> Pricing dinámico · Portfolio P&L; · Briefings IA · FastAPI + Next.js

![Demo](screenshot-url)](loom-url)

## El salto de V1 a V2
[tabla comparativa V1 vs V2 – 5 filas clave]

## Arquitectura
![arch](docs/architecture_v2.png)
[diagrama texto del flujo Backend → DB → IA → Frontend]

## Stack
Backend: FastAPI · SQLAlchemy 2.0 · Alembic · PostgreSQL · Pydantic v2
Frontend: Next.js 14 · TailwindCSS · Recharts · shadcn/ui
IA: Anthropic Claude Haiku (extracción) + Sonnet (briefings)
Deploy: Railway (backend) + Vercel (frontend)

## Demo en vivo
- [App](https://lri-v2.vercel.app)
- [API docs](https://lri-v2.up.railway.app/docs)
- [Video 3min](loom-url)

## Capturas
[3-4 imágenes destacadas: Market Overview, Portfolio, Set Intelligence, Briefings]

## Cómo ejecutar localmente
[instrucciones backend + frontend separadas]

## V1 → V2: decisiones clave
[3 decisiones de producto que explican el salto]

## Limitaciones conocidas
[honestas – precio BrickLink manual, single user, etc.]

## Roadmap V3 (si aplica)
[lo que vendría después]

```

★ EL ASSET MÁS DIFERENCIADOR DE V2

La URL pública de la API con Swagger. Es el único activo de portfolio que demuestra que construiste una API real, documentada, que responde. Ponla en el README, en LinkedIn, en el CV si cabe. Ningún dashboard de Streamlit puede competir con eso visualmente.

Cómo contar el proyecto V2

V2 tiene una narrativa más poderosa que V1 porque tiene una historia de evolución: construiste algo, encontraste sus límites, tomaste decisiones estratégicas, y construiste algo mejor. Eso es exactamente lo que buscan los roles de producto y operaciones.

En el CV — 3 bullets actualizados

	Texto del bullet
Bullet 1 (producto)	Diseñé y construí LEGO Resale Intelligence V2, plataforma de e-commerce intelligence con motor de pricing dinámico sobre histórico de ventas, tracking de portfolio con P&L; en tiempo real y briefings de mercado semanales generados por IA sobre el mercado de LEGO retirado.
Bullet 2 (técnico)	Arquitectura desacoplada FastAPI + Next.js + PostgreSQL; pipeline de extracción con Claude Haiku, síntesis de inteligencia de mercado con Claude Sonnet, migraciones con Alembic, deploy en Railway + Vercel con CI/CD desde GitHub.
Bullet 3 (impacto + evolución)	Evolución de V1 (Streamlit · evaluador puntual) a V2 (SaaS app · motor de inteligencia): reducción del tiempo de análisis de portfolio de >1 hora a <30 segundos; pricing basado en histórico de 300+ sets vs tabla manual de 10 sets.

En LinkedIn — post de evolución (no de lanzamiento)

La diferencia entre un post de V1 y uno de V2 es la narrativa de aprendizaje. V1 fue 'construí X'. V2 es 'construí X, encontré sus límites, tomé estas decisiones, construí Y'. Eso es 10x más valioso para un perfil de producto.

Estructura recomendada (350–450 palabras)

- Hook (1 línea): 'Terminé V1 y me di cuenta de que respondía la pregunta equivocada.'
- La tensión (3 líneas): Qué hacía V1, cuál era su límite real.
- La decisión estratégica (3 líneas): Cambiar la pregunta, no añadir features.
'El salto no fue hacer más cosas — fue hacerlas en el orden correcto.'
- El stack (2 líneas): FastAPI + Next.js. Por qué importó arquitectonicamente.
- Las 3 cosas nuevas que V2 hace (3 líneas): pricing dinámico, portfolio P&L;, briefings IA.
- Aprendizaje honesto (3 líneas): Lo que más te costó. Lo que cambiarías.
(la honestidad transmite madurez y diferencia de perfiles que solo muestran éxitos)
- Links (1 línea): App · API docs · Repo · Video
- Capturas: Market Overview + Portfolio + Briefing + Set Intelligence (4 imágenes)

En entrevista — narrativa de 120 segundos

Sección	Frase 1	Frase 2	
Contexto V1	'Construí una herramienta en Python que analizaba listings de LEGO descatalogado '	'y calculaba si valía la pena comprarlos. Pipeline con Claude Haiku, SQLite, Streamlit.'	
El límite encontrado	'El problema que vi es que respondía una sola pregunta.'	"¿es este listing bueno?" Pero no sabía nada del mercado. Sin historial. Sin tendencias.'	
La decisión de V2	'Decidí cambiar la pregunta. De evaluar un deal a entender el mercado.'	'Eso implicó cambiar arquitectura: FastAPI separado de Next.js, PostgreSQL con histórico.'	
Qué hace V2	'Ahora el sistema tiene un motor de pricing dinámico con datos históricos.'	'un portfolio tracker con P&L; en tiempo real, y Claude Sonnet generando'	'briefings semanales de qué está pa en el mercado.'
Resultado concreto	'Lo que antes tardaba una hora en calcular manualmente — el estado de mi portfolio —'	'ahora lo veo en 30 segundos. Y tengo una URL de API pública documentada.'	
Cierre	'Está deployado. Puedo mostrarte el sistema en vivo si quieres.'	'¿Quieres que entremos en la arquitectura, en las decisiones de producto o en el stack?'	

Adaptar según el rol al que aplicas

Rol	Qué destacar
PM / Producto	Foco en: por qué cambiaste la pregunta de V1. Cómo decidiste el scope de V2. Qué dejaste fuera y por qué. Cómo medirías éxito.
Data / Analytics	Foco en: motor de pricing dinámico. Cómo construiste el historial. Cómo detectas anomalías. Cómo validas la calidad del precio estimado.
Full stack / Ingeniería	Foco en: arquitectura desacoplada. Por qué FastAPI + Next.js. Alembic y migraciones. Tests de integración con httpx async.
Consultoría / Generalista	Foco en: end-to-end ownership. De problema de mercado real a producto funcional. Cómo comunicarías este sistema a un cliente no técnico.
Startup (early stage)	Foco en: podría ser un producto real. Qué validarías antes de lanzarlo. Cuál sería el modelo de negocio. Cómo escalarías los datos de precio.

■ FRASES QUE EVITAR AL CONTAR V2

'Lo hice en dos semanas' — no lo digas aunque sea verdad. Devalúa. 'La IA hace todo el trabajo' — tú diseñaste el sistema. La IA es una herramienta. 'Es como Bloomberg pero para LEGO' — hyperbole. Genera expectativas que no puedes cumplir. 'Podría escalar a millones de usuarios' — nadie te preguntó eso. 'V1 quedó obsoleta' — V1 es la base. Sin V1 no hay V2. Honra el proceso.

Riesgos y errores a evitar en V2

V2 tiene todos los riesgos de V1 más los propios de un stack más complejo. Esta sección es la lista negra actualizada. Léela antes de empezar cada fase.

Errores de producto

Error	Por qué duele	Antídoto
Querer V2 = V1 + TODO lo que quedó fuera	V2 tiene su propio scope acotado. Más ambicioso pero igualmente limitado.	Define la línea de meta de V2 en el día 1. Post-V2.txt para el resto.
Construir el frontend antes de tener datos reales	Un Next.js con mock data no demuestra nada. Es un mockup.	Completa Fases 1–4 antes de tocar Next.js.
Precio dinámico sin datos suficientes para validarlo	Si solo tienes 2 snapshots por set, el 'motor de pricing' es peor que la tabla manual de V1.	Mínimo 5 snapshots por set antes de activar el pricing dinámico en producción.
Portfolio P&L; sin compras reales	Si los datos de portfolio son todos inventados, no puedes validar el sistema.	Usa tus compras reales. O genera seed data muy realista y doc qué es seed y qué es real.
Briefings que no añaden información accionable	Si el briefing dice 'el mercado estuvo activo esta semana', no sirve.	Itera el system prompt hasta que el briefing diga algo que cambiaría una decisión tuya.

Errores técnicos de V2

Error	Por qué duele	Antídoto
CORS mal configurado entre FastAPI y Next.js	El frontend no puede llamar al backend. Bloqueante.	Configura CORS en main.py desde el día 1. Testea con fetch desde el browser.
Variables de entorno distintas en local vs Railway vs Vercel	Funciona en local, falla en producción.	.env.example documenta TODAS las variables. Cada entorno tiene su .env completo.
Migraciones Alembic aplicadas en orden incorrecto	La DB de producción queda en estado inconsistente.	Nunca edites una migración ya aplicada. Siempre crea una nueva.

Async/await inconsistente en FastAPI	Mezclar sync y async en SQLAlchemy causa deadlocks.	Todo async. Si Codex genera algo sync en el contexto de la DB, pídele la versión async.
TypeScript 'any' en el frontend	Pierdes la seguridad de tipos. El frontend no sabe qué viene de la API.	Define tipos en lib/types.ts antes de escribir componentes. No uses 'any'.
Recharts sin ResponsiveContainer	Los gráficos tienen tamaño fijo y no se adaptan al layout.	Siempre envuelve en ResponsiveContainer con width='100%'.
Sin rate limiting en las llamadas a Sonnet	Un bug de bucle infinito puede generar facturas imprevistas.	Límite de 10 llamadas a Sonnet por hora en el código. Log de tokens consumidos.

Errores de portfolio específicos de V2

Error	Por qué duele	Antídoto
README sin las dos URLs (app + API docs)	El reclutador no sabe que hay una API pública.	Dos links en el hero del README: App y API docs. Siempre.
Video demo que muestra Streamlit de V1 en lugar de Next.js de V2	Si el video es de V1, todo el esfuerzo de V2 es invisible.	Graba el video cuando V2 esté completo. No antes.
Presentar V2 sin mencionar V1	Pierdes la narrativa de evolución, que es el activo más diferenciador.	Siempre cuenta V1 → límites → decisión → V2. El viaje vale más que el destino.
Capturas con datos mock obvios	'Portfolio Item 1 · 100€ · 50% profit' → nadie lo cree.	Usa nombres reales de sets. Precios de mercado reales. Fechas coherentes.
No mencionar el costo de Sonnet en la documentación	Parece que ocultas costes. O que no eres consciente de ellos.	Añade en el README: 'Coste estimado por briefing: ~0.05€. ~5€/mes en uso normal.'

■ LA TRAMPA SILENCIOSA MÁS PELIGROSA DE V2

V2 tiene el doble de superficie que V1. El riesgo de 'casi listo' es el doble. Siempre habrá un componente más, un endpoint más, un chart más. Define la línea de meta de V2 hoy: qué 6 pantallas, qué endpoints, qué datos de demo. Cuando esa línea esté cruzada, publicas. No hay V2.1 antes de que V2 sea público.

El V2 que publiques, aunque sea imperfecto, vale infinitamente más que el V2 perfectamente planificado que nunca se enseña.

Construye algo que puedas defender con orgullo en cualquier mesa.